Automate Full Stack Application (MERN)
Project 01

Setup Environment

- After creating project folder I create a **client** and **server** folder for frontend and backend
- Then I go to server folder using **cd server** and install **npm init -y** for initialize node project
- After that I run **npm install express mongoose cors dotenv** those are the dependencies I want
    1. Express is like the **engine of your backend**. It helps your Node.js app **handle requests and responses** easily.

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(5000);
```

    2. Mongoose is a **tool to talk to MongoDB in a structured way. ODM = Object Data Modeling** → it lets you define schemas for your data, so your database knows what "shape" your data should have

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

const User = mongoose.model('User', userSchema);
```

3. **.env file** stores secrets or configs **separately from code**, like API keys, DB URLs, or port numbers.
   require('dotenv').config(); - must import env file like this in server file

4. **CORS = Cross-Origin Resource Sharing**
   Browsers block frontend code (React) from calling a backend on a different port/domain unless you allow it.

- After that I create models folder inside server folder and inside it I create user.js file for MongoDB. It tells MongoDB **what a user looks like** (name, email, age)

- Then I create a routes folder inside sever folder and create routeUser.js file and its
  a. Take the data from frontend
  b. Create a new user object
  c. Save it in MongoDB
  d. Send a response back

Frontend Setup (Client)

- After go to clint folder I run **npx create-react-app .** for install react inside folder
- Then I run **npm start**
- Then I setup **app.js** file

```
import { useEffect, useState } from "react";

function App() {
  const [message, setMessage] = useState("");

  useEffect(() => {
    fetch("http://localhost:5000/user/test")
      .then((res) => res.text())
      .then((data) => setMessage(data));
  }, []);

  return (
    <div>
      <h1>{message}</h1>
    </div>
  );
}

export default App;        You, 1 minute ago • Un
```

- import { useEffect, useState } from "react";
    a. useEffect means Runs code automatically when page loads.
    b. useState means Stores data inside UI.
- I designed UI sepratly so then I can maintain UI easily and connect it to main file (app.js)

Backend Setup (Server)

- In backend specially I created **model folder** and inside it I create **task.js** file. Its mainly focused what is structure of data I enter
- And also I create **route folder** inside it I created **taskroutes.js** file inside it. Its mainly focused on about routes. When I send request to backend **(server.js)** its find routes in **taskroutes.js** file then after connect **mongodb**, update database and give response back to the clint

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URI, {
}).then(() => console.log('MongoDB connected'))
  .catch(err => console.log(err));

const userRoutes = require('./routes/taskRoutes');
app.use('/api', userRoutes);  // all user routes under /api/users

app.listen(process.env.PORT || 5000, () => console.log('Server running'));
```

Figure 01: Server.js file

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema({
  text: { type: String, required: true },       // task description
  category: {
    type: String,
    enum: ['Personal', 'Professional'],
    default: 'Personal'
  },
  completed: { type: Boolean, default: false }, // is task done?
  createdAt: { type: Date, default: Date.now }  // timestamp
});

const Task = mongoose.model('Task', taskSchema);
module.exports = Task;
```

Figure 02: Task.js

```
const express = require('express');
const router = express.Router();
const Task = require('../models/task'); // use Task model now

// ✅ Get all tasks
router.get("/tasks", async (req, res) => {
  try {
    const tasks = await Task.find();
    res.json(tasks);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ✅ Add new task
router.post('/add', async (req, res) => {
  try {
    const { text, category } = req.body;
    const newTask = new Task({
      text,
      category: category || 'Personal'
    });
    await newTask.save();
    res.status(201).json({ message: 'Task added successfully', task: newTask });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Figure 03: Routestask.js file (half a code)

Docker Containerzation

- 1st of all I created dockerfile inside client folder

```
1   FROM node:20-alpine AS builder
2
3   WORKDIR /app
4
5   # Install dependencies first for better layer caching.
6   COPY package*.json ./
7   RUN npm install
8
9   COPY . .
10  RUN npm run build
11
12  FROM nginx:1.27-alpine
13
14  # Remove default nginx static content.
15  RUN rm -rf /usr/share/nginx/html/*
16
17  COPY --from=builder /app/build /usr/share/nginx/html
18
19  EXPOSE 80
20
21  CMD ["nginx", "-g", "daemon off;"]
22
```

- Line number 01, **AS builder** means this stage name is **builder**
- Line number 03, Sets working folder inside container
- Line number 10, This creates React production build
- Line number 12, Instead of Node.js, now it uses **Nginx web server**. This keeps image smaller
- Line number 15, **Deletes default Nginx welcome page files**. Normally Nginx shows: Welcome to nginx. (This removes that.)
- Line number 17, This copies built React files from first stage. **From** /Client/build
  To: **/usr/share/nginx/html (This is Nginx's website folder)** So Nginx now serves your React app
- Tells Docker: This container uses **port 80.** That is HTTP default port.
- Starts Nginx - **`daemon off`** means: keep nginx running in foreground because of Docker needs one main process running. So, container stays alive
  (Simple with daemon off terminal is showing nginx running, by default nginx running in background and terminal is free. Its not suit for this because container want to see at least one running process, so with daemon off fix this, this ensure nginx running on foreground and do not go to background)

- After client docker file I created docker file for server

```
1   FROM node:20-alpine
2
3   WORKDIR /app
4
5   ENV NODE_ENV=production
6
7   COPY package*.json ./
8   RUN npm ci --omit=dev
9
10  COPY . .
11
12  EXPOSE 5000
13
14  CMD ["node", "server.js"]
15
```

- Line number 5 Sets **NODE_ENV** to production. This tells Node & packages like Express that you are in **production mode**
- npm ci = clean install based on package-lock.json
- --omit=dev = skips **devDependencies** (only installs production packages). This keeps the image smaller and lean for production.
- Line number 12, Tells Docker that your app will use port **5000**. This does not publish the port to host automatically; just informs Docker.

- After I create a docker-compose.yml file

```yaml
services:
  ▷Run Service
  server:
    build:
      context: ./server
    container_name: project01-server
    restart: unless-stopped
    env_file:
      - ./server/.env

  ▷Run Service
  client:
    build:
      context: ./client
    container_name: project01-client
    restart: unless-stopped
    depends_on:
      - server

  ▷Run Service
  nginx:
    image: nginx:1.27-alpine
    container_name: project01-nginx
    restart: unless-stopped
    ports:
    - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - client
      - server
```

- restart: unless-stopped means **Docker will automatically restart the container if it crashes, But won't restart if you manually stop it.**
- in nginx server volumes section means**,** Mounts your local Nginx config file **(./nginx/default.conf)** into the container
- :ro = read-only (Nginx can read it but not change it)
- This allows you to customize Nginx behavior (like proxy to your backend/frontend)
- After I hide this ports using env file, I created env file on root folder and type
  NGINX_HOST_PORT=80
  NGINX_CONTAINER_PORT=80
- Then I change docker compose

```yaml
ports:
    - "${NGINX_HOST_PORT}:${NGINX_CONTAINER_PORT}"
```

NGINX Configuration

- Finnaly I created **default.config** file in the ngix folder

```
server {
    listen 80;

    location / {
        proxy_pass http://client:80;
    }

    location /api/ {
        proxy_pass http://server:5000;
    }
}
```

- This starts one server block. Think of it like one website rule set
- Nginx listens on **port 80**

  <u>Frontend part</u>

  a. if request starts with / (example: /app | /task) send it to: http://client:80

  <u>Backend part</u>

  a. if request starts with /api/ (example : /api/tasks | /api/users) send it to: http://server:5000

- **FLOW DIAGRAME**

```
Browser request
      ↓
 localhost:80
      ↓
    Nginx
   ↙      ↘
 /          /api
 ↓            ↓
client:80   server:5000
```

✅ Why this is useful

Without Nginx:

- frontend = `localhost:3000`
- backend = `localhost:5000`

With Nginx:

- frontend = `localhost`
- backend = `localhost/api`

✅ looks professional
✅ single port
✅ easier deployment
✅ frontend and backend feel like one app

Jenkins pipelines Setup

- **Checkout stage**
  What checkout actually does
  a. **connects to GitHub repo**
  b. **pulls latest code**
  c. **checks out the branch**
  d. **places files inside Jenkins workspace**
  e. **makes code ready for next stages**

```
stages {
    stage('Checkout') {
        steps {
            git branch: 'main',
                url: 'https://github.com/sachilz/dockerized-mern-app'
        }
    }
```

- **List stage**
  In this stage list all file and folder inside our project folder

```
        stage('List Files') {
            steps {
                bat 'dir'
            }
        }
```

- **Gitleaks Stage - 01**
  This is 1st security layer, in this layer do Secret Scanning (SAST Security Layer)
  a. Accidentally pushed .env
  b. JWT secrets
  c. MongoDB credentials
  d. API key
- In the top we must indicate gitleaks file path and then assign it gitleaks stage

```
environment {
    GITLEAKS_PATH = "C:\\gitleaks\\gitleaks.exe"
}
```

```
stage('Gitleaks Secret Scan') {
    steps {
        bat "${GITLEAKS_PATH} detect --source . --verbose"
    }
}
```

- **SonarQube Stage (SAST) – 02**
  What is SAST means : Because it is doing code checking before running the app
  SonarQube looks at it like: Let me check if this code is safe or has mistakes
  SAST = Static Application Security Testing

  So,
  SAST - checks code before running
  DAST - checks app while running

  This check
  a. bugs
  b. security vulnerabilities
  c. bad coding practices
  d. code smell
  e. maintainability issues

How to setup SonarQube

- First of all, ensure u install java version 21 - Download Link
- Then u muse download sonarqube community build - Download Link
- U must download then sonarscanner also (CLI) - Download Link
- Then extract the sonarqube community setup and move it DISK C, then go to inside that folder and go **bin** folder and then **windows-x86-64** folder inside it has **StartSonar.bat** file. Open it and wait 1-2 minute

- After that u can go to browser and type **http://localhost:9000** and login by typing user name as **admin** and password as **admin**. Then u can change ur password If want.
- After that u must create a token (IMPORTANT) to do that go to ur account (TOP RIGHT) u can see profile icon and go to my account under it select security and then u can create token.
- Give any name for token and use expiration as NO EXPIRE and type as PROJECT ANALYSIS TOKEN then create. **Copy it and later u can't see it**
- Now go to Jenkins and install plugins **(SonarQube Scanner)** install it
- Then go to Jenkins setting and go credentials click add credentials after u must select **secret text**. Scope must be global and in secret section paste that copied token from SQ
- In the id section u must add anything u want (RECOMMAND : use small word) because we wanna it later.
- Then extract CLI Package and paste it into DISK C and go to inside that folder then go to bin folder and u see **SonarScanner.bat** file
- Go to inside Jenkins setting and click system. Scroll to bottom find **SonarQube servers**
    a. Tick Environment variables Square
    b. Name as "**sonarqube**" – we must this one later
    c. Server URL as http://localhost:9000
    d. Server authentication token add token from credential we add it earlier
- Then after doing those things apply and save

```
        stage('SonarQube Scan') {
            steps {
                withSonarQubeEnv('sonarqube') {
                withCredentials([string(credentialsId: 'sonar-token', variable: 'SONAR_TOKEN')]) {
                    bat """
                    \"${SONAR_SCANNER}\" ^
                    -Dsonar.projectKey=mern-todo-docker-nginx-deployment ^
                    -Dsonar.sources=. ^
                    -Dsonar.host.url=http://localhost:9000 ^
                    -Dsonar.token=%SONAR_TOKEN%
                    """
                }
            }
        }
    }

        stage('Quality Gate') {
            steps {
                timeout(time: 20, unit: 'MINUTES') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }
    }
}
```

Code Explanation

- **Line number 31** - connects Jenkins to your SonarQube server. Must match **sonarqube** as exact name when u add ID as Jenkins credentials
- **Line number 32** - safely gets your Sonar token from Jenkins secrets. I will explain how I get that line (that line created by Jenkins)
  withCredentials([string(credentialsId: 'sonar-token', variable: 'SONAR_TOKEN')])
    a. Go to ur Jenkins pipelines build page and click a config
    b. After go to inside it go pipeline section and bellow that u can see pipelines syntax click it
    c. In Sample Step click and drop down it then u must find **with credential** label and select it
    d. In Bindings add secret text and set variable as u can any name and Credentials as u must add earlier, **we add token as credentials u can add it this**
    e. Then create **pipeline generate script** button then u can copy that code line and put ur code
- **Line number 33 -** runs the scanner command on Windows (Jenkins send my project code to SonarQube for checking)
- **Line number 34 -** This is the path to SonarScanner. We added top of the pipline

```
environment {
    GITLEAKS_PATH = "C:\\DevSecOps\\gitleaks\\gitleaks.exe"
    SONAR_SCANNER = "C:\\DevSecOps\\sonar-scanner\\bin\\sonar-scanner.bat"
}
```

- **Line number 35 -** This is your project name / ID in SonarQube dashboard. (u can change it)
- **Line number 36 -** Sonar scans all code in current Jenkins workspace. This **“.”** Means Current folder
- **Line number 37 -** This tells SonarScanner where your SonarQube server is running.
- **Line number 38 -** This is the password-like secret token. It allows Jenkins to send code to SonarQube securely
- **^ means -** This is only for Windows multi-line command continuation. It means continue command in next line **Without ^,** CMD thinks command ended. So all lines become **one single scanner command**.

Quality Gate Stage

- **Line number 47 -** wait maximum **20 minutes**. Jenkins waits for SonarQube result. Because SonarQube needs some time to analyze your code.
  (I Faced issue in this section)
    a. I run it but not completed because Jenkins wait for sonarqube result. But Jenkins don't get it. Because between Jenkins and SQ dosen't have proper communication.
    b. So I use solution as Webhook in the sonarqube
    c. How I do it, Lets look at the that thing
        - Go to sonarqube and then go to **administration** section after left panel u see **Configuration**, under it u see **webhook**
        - When u add webhook u must use **engrok** (Because http not supported webhook)
        - Sample URL – http://localhost:8080/sonarqube-webhook/
        - After NGROX it look like this – https://xyz.124/sonarqube-webhook/
        - This **/sonarqube-webhook/** is unique
- **Line number 48** – wait for the SQ report if that Pass or fail. **abortPipeline: true** means stop everything if quality is bad

Hadolint Scan

```
54        stage('Hadolint Scan') {
55            steps {
56                bat '''
57                docker run --rm -i hadolint/hadolint < client/Dockerfile
58                docker run --rm -i hadolint/hadolint < server/Dockerfile
59                '''
60            }
61        }
```

- This layer check dockerfiles
- Hadolint scans Dockerfiles for
    a. insecure base image usage
    b. latest tag usage
    c. root user issues
    d. apk/apt cache issues
    e. missing WORKDIR
    f. unnecessary sudo
    g. bad COPY usage
    h. layer optimization

- **Line number 56** - your Jenkins runs on **Windows**, **bat** means: Run Windows batch commands. If Linux → use **sh**
- **Line number 57 -** This runs the official Docker image hadolint/hadolint
    a. **docker run** - Start a temporary container
    b. **-- rm** - Delete container automatically after scan. So Jenkins won't keep unnecessary stopped containers.
    c. **-i** - Interactive input mode. Means **pass Dockerfile content into Hadolint container through STDIN**
    d. **hadolint/hadolint -** first hadolint → **Docker Hub repository** | second hadolint → **image name**
    e. **< client/Dockerfile -** Open client/Dockerfile **then** send all its text into the Hadolint container **after** Hadolint scans that content

Inject Env File

```
63        stage('Inject Env File') {
64            steps {
65                withCredentials([
66                    file(credentialsId: 'todo-app-server-env', variable: 'SERVER_ENV')
67                    ]) {
68                    bat "copy \"%SERVER_ENV%\" server\\.env"
69                }
70            }
71        }
```

- In this stage we must add env file for our Jenkins credentials if not our backend or env related files are not work
- After adding credentials, we have to create a pipelines syntax using Jenkins configuration. I created a that syntax and line number 65 – 67 is showing that syntax
- **Line number 68** - copy the secret **.env** file from Jenkins credentials into your project's **server/.env.** This is how your backend gets environment variables securely during pipeline runtime

Trivy Base Image Scan

```
73        stage('Trivy Base Image Scan') {
74            steps {
75                bat "\"${TRIVY_PATH}\" image --severity HIGH,CRITICAL node:20-alpine"
76                bat "\"${TRIVY_PATH}\" image --severity HIGH,CRITICAL nginx:1.27-alpine"
77            }
78        }
```

- So this stage checks whether the **foundation of your containers is already vulnerable**. Trivy analyzes image layers, installed OS packages, and metadata, and filters to only HIGH/CRITICAL findings
- Check whether these **official parent images (node:20-alpine / nginx) already contain CVEs**
- **Line number 75** - This scans the official Node.js Alpine base image**.** It checks
    a. Alpine OS packages
    b. OpenSSL
    c. Busybox
    d. Libc
    e. Node runtime packages
    f. package manager libraries
- **Line number 76** - This scans the official Nginx reverse proxy base image. It checks
    a. nginx binary
    b. Alpine packages
    c. OpenSSL
    d. SSL libraries
    e. OS packages

Docker Compose Build

```
80        stage('Docker Compose Build') {
81            steps {
82                bat 'docker compose build'
83            }
84        }
```

- In this stage building docker images and container using docker compose file

Trivy final image Scan

```
86        stage('Trivy final image Scan') {
87            steps {
88                bat "\"${TRIVY_PATH}\" image --severity HIGH,CRITICAL todo-app-server"
89                bat "\"${TRIVY_PATH}\" image --severity HIGH,CRITICAL todo-app-client"
90            }
91        }
```

- In this stage scan each container-based image (Final scan)
- **Line number 88-89** scan server image and client image

Run Containers

```
93        stage('Run Containers') {
94            steps {
95                bat 'docker compose up -d'
96            }
97        }
```

- **-d** means **detached mode.** So when run this terminal become free process running foreground

Full Jenkins Pipeline

https://github.com/sachilz/CI-CD-Enabled-MERN-Stack-Todo-Application/blob/main/Jenkinsfile

Deploy On AWS

- First of all, at the beginning we run this 1 EC2 instance
- So, all files go to 1 EC2 instance
- So, to do this create EC2 instance and use free tier (t3.micro)
- After change ports (for my one – Nginx port 80 and Backend Port 500) setup it
- To setup it clicks EC2 instance and go security section under this edit inbound ports. Add custom port as 80 and 5000 then save
- When creating instance must be create keypair and save it
- Then create a folder inside ur computer and put that .pem file into it
- Then open that folder through vscode and then open terminal
- Type this ssh -t (keypare name – demo.pem) ubuntu@(ur instance public ip)
  ssh -i ubuntu@56.89.21.135
- Then u can go to insider server
- After going inside server setup, it like this (this is depended on ur project dependencies)
  a. Crate a folder inside server – **mkdir project1**
  b. Then update ur package **– sudo apt update**
  c. Install git **– sudo apt install git -y**
  d. Install node **- sudo apt install -y nodejs** {if this install old node version then u must download ur preferd version}
     - To do this first check ur node version using **node -v**
     - If its old, run these command
       curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
       source ~/.bashrc
       nvm install 21
       nvm use 21
     - Then check again **node -v**
  e. Install npm – **sudo apt install npm**
- Done now u have to clone ur project – **sudo git clone <repo link>**
- Now go to ur folder (for me I have sever and client file) first of all I go to server folder and run **npm install** then go to client folder and run **npm install.** This create nodemodules folder inside each folder
- One more thing if u use .env file in this instance its missing because its ignore by gitignore. We have to setup it like this go to ur env file folder (for me I setup it server folder) go to server folder - **cd server**
- Then create .env file using **touch .env**
- Now edit **nano .env**
- Now u can write inside it all of this and after write press **CTRL + X** and then **Press Y** then **ENTER**

- If u want to install nodemon then run this **npm install -g nodemon (OPTIONAL)**
- Now u can run docker compose **- sudo docker-compose up -d –build**
- Now u can up docker **docker-compose up -d –build**
- Why we use –build again
    - Rebuild your images from Dockerfile
    - Reinstall dependencies if needed
    - Pick up latest code changes
    - Then start containers